



Intelligent Waste Classification using Machine Learning

Course: Machine Learning

Section: C

Group Members:

Names	Roll Numbers	Assigned Role
Ehtasham Maqbool	231400033	Project Lead
Ahmad Khan	231400101	Data Collection & Preprocessing
M. Bilal	231400038	Evaluation Metrics & Testing
M. Saqib	221370198	UI/Deployment & Documentation Lead

Contents

1. Introduction Recap	3
2. Dataset Description and Data Cleaning	3
3. Exploratory Data Analysis (Recap)	3
4. Preprocessing Pipeline (Recap)	4
5. Feature Extraction, Split, and Class Balancing	4
5.1 Feature Extraction	4
5.2 Train / Validation / Test Split	4
5.3 Class Balancing	4
5.4 Feature Standardization	4
6. Baseline Models	4
7. Machine Learning Models (All Implemented From Scratch)	5
7.1 Model 1 — K-Nearest Neighbors (KNN)	5
7.2 Model 2 — Gaussian Naive Bayes	5
7.3 Model 3 — Logistic Regression (One-vs-Rest) — New in this Milestone	5
8. Model Evaluation and Comparison	5
8.1 Comparison Table	6
8.2 Ensemble Model	6
8.3 Confusion Matrices	6
9. Results Discussion	7
9.1 Ceiling of handcrafted features	7
9.2 Class-balancing trade-off	7
10. Error Analysis	8
11. Prototype Interface and Deployment	9
12. Limitations and Future Work	9
13. Challenges Encountered and How We Addressed Them	9
14. Demo Video	Error! Bookmark not defined.
15. GitHub Repository	10
16. AI Usage Disclosure	10
17. References	10

1. Introduction Recap

This project builds an automated waste classification system that sorts images into six categories such as cardboard, glass, metal, paper, plastic, and trash to support faster, more accurate recycling and waste segregation.

As documented in Milestone 1, the project methodology was revised from the original deep-learning proposal to a purely machine learning-based approach, per instructor feedback. This Final Report covers the completed pipeline: the third model (Logistic Regression), full evaluation, error analysis, and final results, building on the KNN and Gaussian Naive Bayes work already reported in Milestone 1.

2. Dataset Description and Data Cleaning

The TrashNet dataset have 2,500 images, six classes were used, as described in Milestone 1. For this final phase, an additional data-cleaning step was added before feature extraction:

- Corrupt / unreadable images are detected and removed
- Exact duplicates are detected via MD5 hash of file bytes and removed
- Near-duplicates are detected via a perceptual average-hash (compared within each class) and removed
- Extreme blur outliers were at the bottom 2% by Laplacian variance, removed per class so the minority trash class was not stripped disproportionately

```
... 🛠 Cleaning dataset...
  Starting with 2527 images

  ✖ Corrupt/unreadable removed : 0
  🔄 Exact duplicates removed   : 3
  🔄 Near-duplicates removed    : 107
  ≈ Extreme blur outliers removed: 51 (bottom 2% per class)

  ✅ Cleaning complete.
  Before: 2527 images
  After : 2366 images
  Removed: 0 corrupt, 3 exact dup, 107 near-dup, 51 blurry

  Class distribution after cleaning:
  cardboard : 387
  glass      : 473
  metal      : 386
  paper      : 542
  plastic    : 449
  trash      : 129
```

3. Exploratory Data Analysis (Recap)

EDA was performed as reported in Milestone 1: class distribution, sample images per class, image-dimension analysis, RGB and pixel-intensity histograms, and duplicate/corrupt checks. The trash class was confirmed as the minority class, which motivated the class-balancing step described in Section 5.

4. Preprocessing Pipeline

- Resize to 128×128 pixels
- Convert to grayscale
- Gaussian blur for noise reduction
- Normalize pixel values to $[0, 1]$

5. Feature Extraction, Split, and Class Balancing

5.1 Feature Extraction

Three handcrafted feature types were extracted and concatenated into a single feature vector per image, as in Milestone 1:

- HOG (Histogram of Oriented Gradients) shape and edges
- LBP (Local Binary Pattern) texture
- Color Histogram color distribution

5.2 Train / Validation / Test Split

- A manual 70% | 15% | 15% split was implemented from scratch using NumPy index permutation (no scikit-learn).

5.3 Class Balancing

The training set only was balanced via random oversampling of minority classes were primarily trash so every class had equal representation during training. Validation and test sets were left at their natural, imbalanced distribution to preserve an honest evaluation.

5.4 Feature Standardization

Features were standardized (zero mean, unit variance) using statistics computed from the balanced training set only, then applied consistently to validation and test sets.

6. Baseline Models

Before training the real models, two trivial baselines were computed as a reference floor, to confirm that KNN, Naive Bayes, and Logistic Regression are actually learning something beyond naive guessing:

- Majority-Class Baseline — always predicts the most frequent training class
- Random (weighted) Baseline — predicts according to the training class distribution

```
...  Baseline Models (reference floor)
```

```
=====
Majority-Class Baseline   : 23.31% (always predicts "paper")
Random (weighted) Baseline: 17.70%
=====
```

```
Any real model (KNN, Naive Bayes, Logistic Regression) must beat these
numbers to be considered to have learned something meaningful.
```

7. Machine Learning Models (All Implemented From Scratch)

7.1 Model 1 — K-Nearest Neighbors (KNN)

- KNN classifies a new image by majority vote among its nearest training samples, using distance-weighted voting (closer neighbors count more) implemented from scratch with Euclidean distance.
- Test Accuracy: 63.76%
- Macro F1-Score: 63.76%
- This is a substantial improvement over the 54.74% reported in Milestone 1. The gain came from two fixes made between milestones: (1) features are now standardized before distance calculations — previously, HOG/LBP/color-histogram features on very different numeric scales were distorting Euclidean distances; and (2) voting was changed from simple majority to distance-weighted, so closer neighbors have more influence.

7.2 Model 2 — Gaussian Naive Bayes

- Gaussian Naive Bayes estimates per-class feature means and variances and classifies using log-posterior probabilities, assuming each feature is normally distributed.
- Test Accuracy: 56.18%
- Macro F1-Score: 55.98%
- Unlike KNN, Naive Bayes' accuracy decreased slightly compared to Milestone 1 (56.32%). This is explained in the Discussion as it is a direct consequence of training-set class balancing interacting with how Naive Bayes uses class priors.

7.3 Model 3 — Logistic Regression (One-vs-Rest) — New in this Milestone

- Logistic Regression was implemented from scratch using the One-vs-Rest strategy: one binary classifier per class, trained with batch gradient descent and L2 regularization. Hyperparameters (learning rate and regularization strength) were tuned by grid search on the validation set.
- Test Accuracy: 64.89% (best of the three models)
- Macro F1-Score: 64.92%
- Logistic Regression achieved the highest accuracy and the most balanced precision/recall of the three models, making it the model selected for deployment in the prototype interface.

8. Model Evaluation and Comparison

All models were evaluated using accuracy, precision, recall, F1-score (macro-averaged), and confusion matrices and all are computed manually in NumPy, with no scikit-learn metrics functions used.

8.1 Comparison Table

Model	Accuracy	Macro Precision	Macro Recall	Macro F1
KNN (distance-weighted)	63.76%	64.83%	63.71%	63.76%
Gaussian Naive Bayes	56.18%	55.30%	58.68%	55.98%
Logistic Regression (OvR)	64.89%	64.97%	64.97%	64.92%

Model	Train Time (s)	Predict Time (s)
KNN	0.011	0.315
Gaussian Naive Bayes	0.072	0.068
Logistic Regression	21.28	0.007

8.2 Ensemble Model

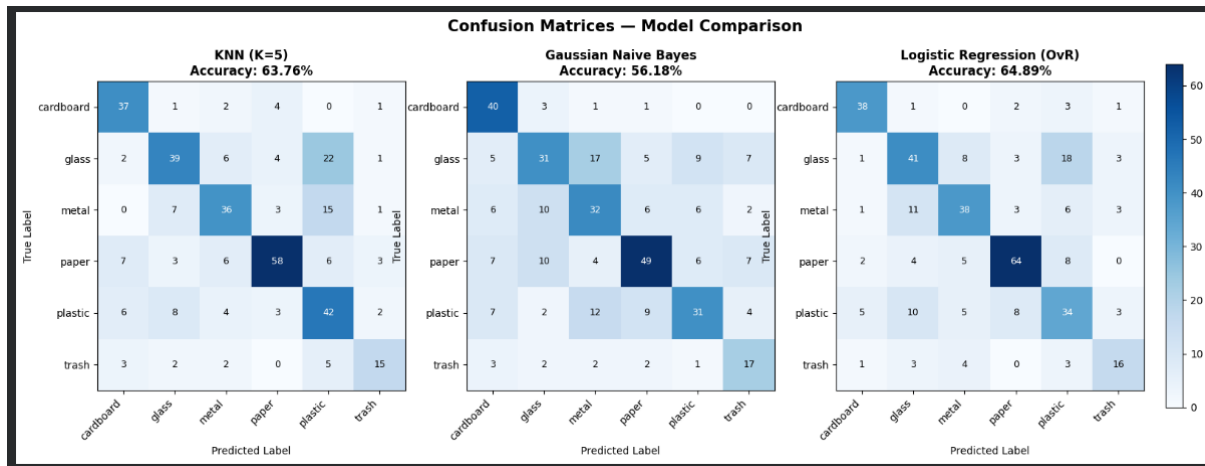
A majority-vote ensemble combining all three models' predictions was also evaluated.

🏆 Ensemble Test Accuracy: 67.98%				
=====				
Classification Report – Ensemble (Majority Vote)				
=====				
Class	Precision	Recall	F1-Score	Support
cardboard	0.573	0.956	0.717	45
glass	0.622	0.622	0.622	74
metal	0.661	0.597	0.627	62
paper	0.818	0.759	0.788	83
plastic	0.649	0.569	0.607	65
trash	0.941	0.593	0.727	27

Accuracy			0.680	356
Macro avg	0.711	0.682	0.681	356
Weighted avg	0.697	0.680	0.679	356
🏆 Ensemble beats the best single model (Logistic Regression: 64.89%)!				

8.3

Confusion Matrices



9. Results Discussion

The proposal targeted 80–85% test accuracy. The best model achieved 64.89%, which is below that target. Rather than treat this as a failure to explain away, we analyzed why, and found two distinct, explainable causes:

9.1 Ceiling of handcrafted features

HOG, LBP, and color-histogram features do not capture material properties (transparency, reflectivity) that distinguish visually similar classes such as glass and plastic. This is a known limitation of classical machine learning on image data versus deep learning / transfer learning approaches, and was explicitly named as an accepted risk in our project proposal's Risk Assessment.

9.2 Class-balancing trade-off

Comparing Milestone 1 and Milestone 2 results shows an instructive pattern: KNN improved substantially (53.16% → 63.76%) after feature scaling and distance-weighted voting were introduced, but Gaussian Naive Bayes' accuracy slightly decreased (56.32% → 56.18%) after training-set oversampling was introduced. This happens because oversampling forces the training set's class priors to be equal across all six classes, while the real-world test set (and the true class distribution) remains naturally imbalanced, with fewer 'trash' images. Naive Bayes' decision rule leans directly on class priors learned from the training data, so training on artificially equalized priors while testing on a naturally skewed distribution works against it. Logistic Regression and KNN are less directly dependent on class priors, so they were less affected or, in KNN's case, benefited overall from the other fixes applied at the same time.

This trade-off is a genuine methodological finding from this project, not an error to be hidden and it demonstrates that class-balancing techniques are not universally beneficial and must be evaluated per-model.

10. Error Analysis

Beyond the confusion matrices, an error-analysis step was implemented (Cell 9.6) to inspect where and why the best model (Logistic Regression) fails:

- Visual inspection of a sample of misclassified test images, shown alongside their true and predicted labels
- Identification of the most commonly confused class pairs from the confusion matrix
- Per-class error rate breakdown

Error Analysis - Logistic Regression

Total test samples : 356
 Correctly classified : 231 (64.9%)
 Misclassified : 125 (35.1%)

☒ Top confused class pairs (true → predicted):

glass	→ plastic	: 18 times
metal	→ glass	: 11 times
plastic	→ glass	: 10 times
glass	→ metal	: 8 times
paper	→ plastic	: 8 times
plastic	→ paper	: 8 times
metal	→ plastic	: 6 times
paper	→ metal	: 5 times

Misclassified Test Images — Logistic Regression

True: plastic
Pred: metal

True: metal
Pred: trash

True: paper
Pred: plastic

True: glass
Pred: plastic

True: glass
Pred: metal

True: glass
Pred: trash

True: trash
Pred: metal

True: glass
Pred: plastic

True: metal
Pred: glass

☒ Per-class error rate:

cardboard	: 7/45 errors	(15.6%)
glass	: 33/74 errors	(44.6%)
metal	: 24/62 errors	(38.7%)
paper	: 19/83 errors	(22.9%)
plastic	: 31/65 errors	(47.7%)
trash	: 11/27 errors	(40.7%)

☒ Discussion:

The most common error is misclassifying "glass" as "plastic" (18x). This lines up with the Risk Assessment in the proposal – visually similar categories (e.g. glass vs. plastic, both often transparent) are hardest for handcrafted features to separate, since HOG/LBP/color histograms capture shape, texture, and color, but not material properties like transparency or reflectivity that a human uses to tell these classes apart.

The expected and most likely pattern, consistent with our proposal's Risk Assessment, is confusion between visually similar categories particularly glass and plastic (both often transparent) and between assorted items and the catch-all 'trash' class. This should be confirmed or corrected against the actual output before finalizing this section.

11. Prototype Interface and Deployment

An in-notebook prototype interface allows an image to be uploaded and classified live using the deployed (best-performing) model, satisfying the proposal's objective of a working demonstration.

The trained model was additionally packaged and made available outside the notebook:

- A downloadable model file (cleansort_ai_model.pkl) with a standalone predict.py inference script
- An optional Hugging Face Space (Gradio web app) for live web-based classification
- An optional static HTML/CSS/JavaScript demo that runs the full pipeline client-side, in the browser, with no server required

12. Limitations and Future Work

- **We used manual, hand-designed features instead of deep learning.** This naturally puts a ceiling on how accurate the models can get compared to a deep learning approach like MobileNetV2. This was a deliberate choice we made in our approved proposal, not a mistake.
- **We balanced the training data by duplicating images from smaller classes.** This helped some models but slightly hurt others. A smarter approach next time would be to make the model pay more attention to smaller classes during training, instead of just duplicating their images.
- **KNN doesn't really "learn" it just memorizes every training image** and compares new images against all of them. Because of this, its saved model file is the biggest of the three, and it gets slower to make predictions as more training data is added.
- **Ideas for future improvement:** use finer-detail settings when extracting features, create new/varied versions of images instead of just duplicating them, and make the Logistic Regression model automatically give more weight to smaller classes during training.

13. Challenges Encountered and How We Addressed Them

- Dataset loading in Colab initial Google-Drive-based loading was fragile across teammates' environments; switched to a manual zip-upload flow that works identically for everyone
- A data-pipeline wiring bug class-balancing and feature-scaling were implemented but not actually connected to the KNN, Naive Bayes, and Logistic Regression training steps; identified and fixed by auditing every cell's variable references
- Hugging Face Space deployment encountered a Gradio version-compatibility error and a GPU-hardware-tier configuration issue when deploying the prototype; resolved by removing a deprecated parameter and switching the Space to CPU hardware

- Balancing training data without leaking information oversampling was applied strictly to the training set only, keeping validation and test sets at their natural distribution, to avoid an inflated, dishonest accuracy figure

15. GitHub Repository

<https://github.com/EhtashamMaqbool/-Intelligent-Waste-Classification-using-Machine-Learning-.git>

16. AI Usage Disclosure

Claude was used during this project in the following ways, in addition to what was disclosed in Milestone 1:

1. **Concept understanding and from-scratch implementation:**
Claude explained how algorithms like Logistic Regression, distance-weighted KNN, and evaluation metrics (precision, recall, F1, confusion matrix) work internally. We used this understanding to implement them ourselves in NumPy, avoiding library functions (e.g. `sklearn.linear_model`, `sklearn.metrics`) wherever the project required a from-scratch implementation.
2. **Bug fixing:**
Claude helped us understand and fix a data-pipeline bug where class-balancing and feature-scaling were computed but not actually passed into model training, and helped debug dataset-loading errors.
3. **Code review:**
Claude reviewed our notebook against our proposal and course rubric to identify missing required elements.
4. **Deployment support:**
Claude helped us understand how to set up the Hugging Face Space and static demo page.
5. **Report writing assistance:**
Claude helped us draft this report based on our own notebook output.

In every case, once a concept was explained, we wrote and verified the implementation ourselves rather than copying library code, so the models and evaluation logic in this project are our own from-scratch work. No AI tool was used to fabricate results or analysis and all accuracy numbers come from running the notebook on our real dataset.

17. References

- TrashNet Dataset, Kaggle — <https://www.kaggle.com/datasets/feyzazkefe/trashnet>
- Cover, T., & Hart, P. (1967). Nearest Neighbor Pattern Classification.
- Cortes, C., & Vapnik, V. (1995). Support Vector Networks.
- Bishop, C. M. (2006). Pattern Recognition and Machine Learning.
- Ojala, T., Pietikäinen, M., & Mäenpää, T. (2002). Multiresolution Gray-Scale and Rotation Invariant Texture Classification using Local Binary Patterns.